# Churn-HPC: Executive Technical Report

DNN churn classification across the modern AI infrastructure stack

Anh Quang Nguyen

July 28, 2026

## 1. Problem and Objective

Customer churn prediction on `Churn_Dataset.csv`: 5,000 telecom customers, 16 numeric usage features, boolean `churned` label with a 14% positive rate. The modeling task (a multilayer perceptron reaching 92-94% test accuracy against an 86% majority baseline) is deliberately modest; the engineering objective is the point: run one workload through an immutable, reproducible infrastructure pipeline spanning containerization, orchestration, storage staging, and multi-hardware benchmarking, and identify the true performance bottleneck with measured telemetry rather than intuition.

The resource challenge is the inverse of the usual one. The dataset is 341 KB; every layer of the stack (10-core CPU node, Apple M4 GPU, provisioned NVIDIA L4 and TPU v5e targets) is oversized for it. That makes it a clean probe of **fixed costs**: compilation, kernel dispatch, staging, orchestration overhead. These are exactly the costs that silently dominate small-to-medium scientific workloads on shared clusters.

## 2. Architecture

**Phase 1 - Immutable environment.** Two Dockerfiles (`docker/`): a CPU image on `python:3.12-slim` and a CUDA 12.4 + cuDNN image carrying the pinned `jax[cuda12]`/XLA and PyTorch stack. All dependencies are baked at build time from pinned requirements files; running nodes install nothing. On SLURM the same images run read-only via Apptainer (`.sif`).

**Phase 1 - Orchestration manifests.** Kubernetes Jobs (`k8s/`) declare exact hardware constraints: the GPU job requests and limits 8 CPU cores, 24 GiB RAM, and exactly 1 `nvidia.com/gpu` on an L4 node pool; the TPU job pins a v5e 2x2 slice (4 chips, 24 cores, 48 GiB). SLURM batch scripts (`slurm/`) mirror this with `--cpus-per-task`, `--mem`, and `--gres=gpu:1`, plus a 2-node `jax.distributed` variant with the coordinator pinned to the first node of the allocation.

**Phase 2 - Provisioning and storage.** Three mapped targets: the local M4 node (measured), the Stanford ME344 SLURM cluster `hpcc-cluster-49` (VPN-gated, manifests ready), and a private VPC-native GKE cluster (subnet `10.10.0.0/20`, pod/service secondary ranges, no public node IPs, egress via Cloud NAT). The data tier funnels the CSV as: shared NFS or GCS canonical copy, staged once per job to node-local NVMe scratch (`$SLURM_TMPDIR` / local-SSD `emptyDir` / gcsfuse file cache), then loaded to RAM once and pushed to device memory. The training loop never touches shared storage.

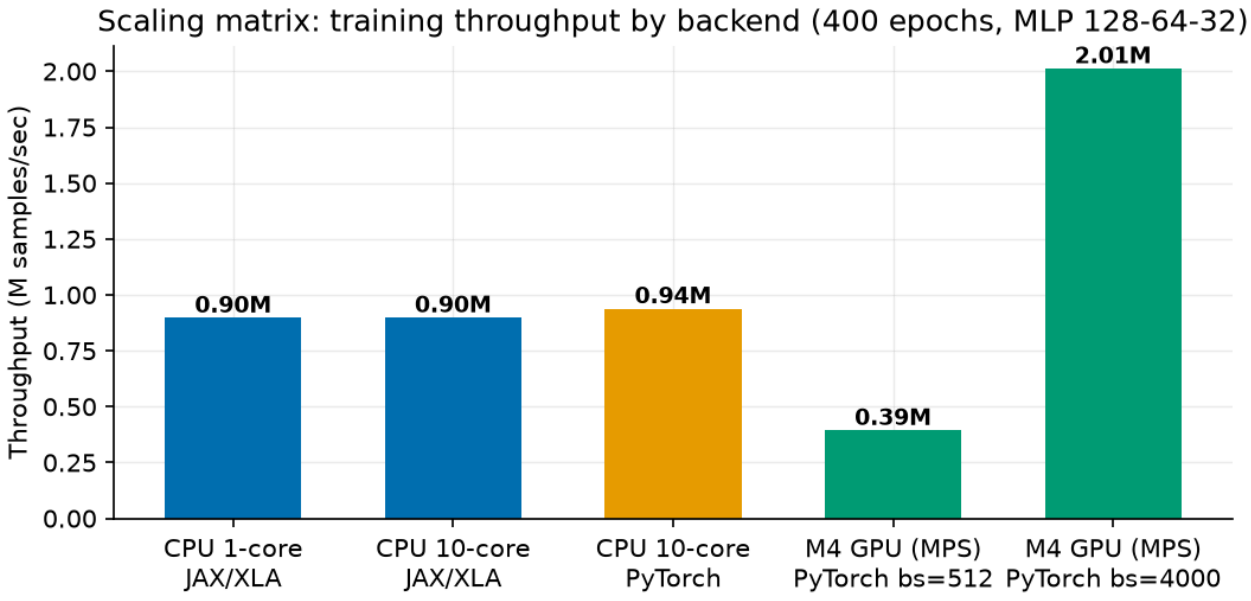
**Model.** MLP 128-64-32 with ReLU, softmax cross-entropy, Adam 1e-3. Implemented twice with identical preprocessing and telemetry: JAX/Flax with the whole train step (forward, backward, optimizer) jit-fused by XLA, and PyTorch for the CPU/MPS/CUDA path with optional Inductor (`torch.compile`).

## 3. Measurements

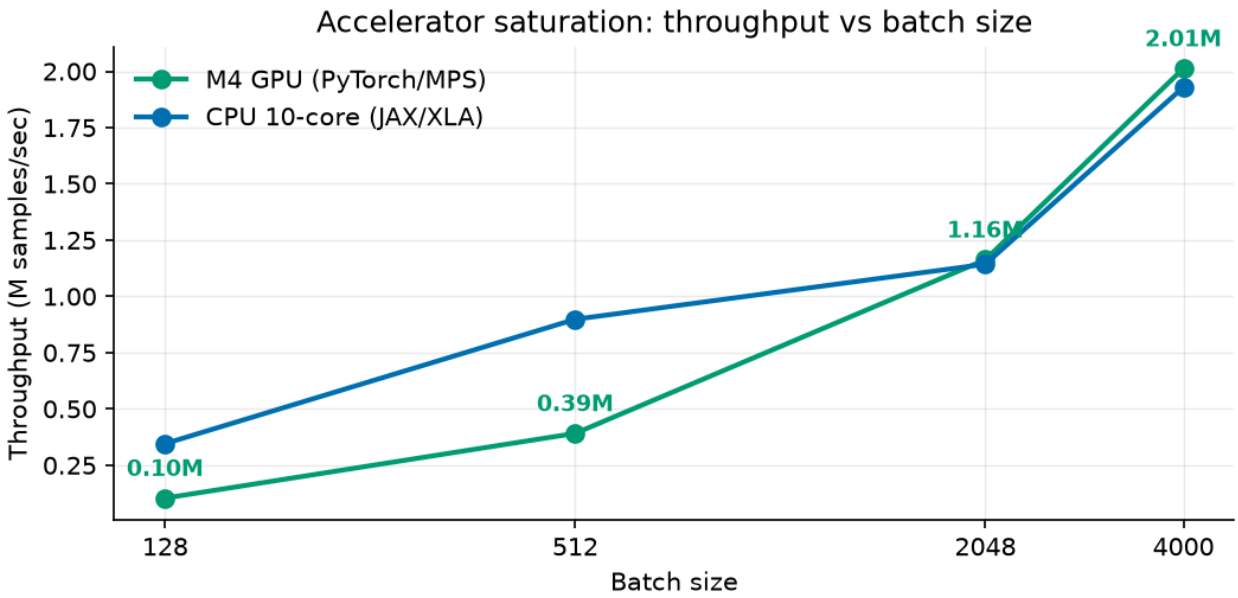
Telemetry is captured by a background sampler (`src/telemetry.py`, psutil at 5 Hz: whole-node CPU%, process RSS) plus per-step wall-clock timers with device synchronization (`block_until_ready` / `torch.mps.synchronize`) so step times are honest. Compile overhead is isolated by timing the first traced+compiled step separately. The SLURM GPU job additionally logs `nvidia-smi` at 1 Hz for the cluster rungs. Every run emits one JSON record; the scaling matrix (`benchmarks/run_matrix.sh`) is 12 configurations at 400 epochs.

## 4. Results: the scaling matrix

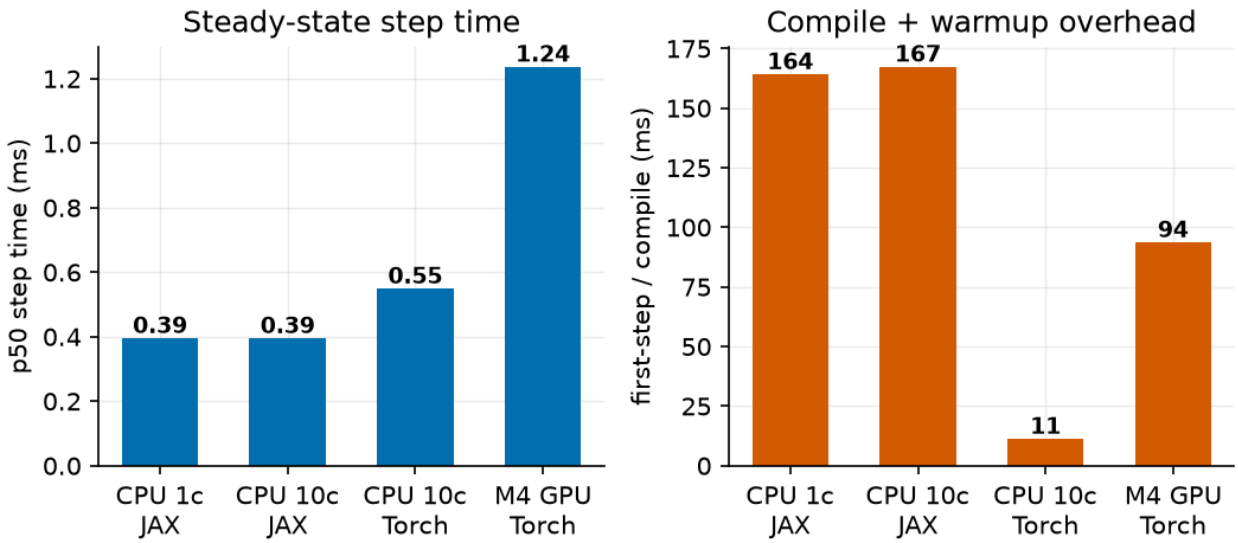
| Config | Compile (s) | p50 step (ms) | Samples/s | Node CPU % | Peak RSS (MB) | Acc |
|---|---|---|---|---|---|---|
| CPU 1-core, JAX/XLA | 0.164 | 0.395 | 897 K | 19.2 | 358 | 0.916 |
| CPU 10-core, JAX/XLA | 0.167 | 0.395 | 898 K | 19.4 | 361 | 0.916 |
| CPU 10-core, PyTorch | 0.011 | 0.549 | 938 K | 17.7 | 312 | 0.917 |
| M4 GPU (MPS), bs 512 | 0.094 | 1.237 | 394 K | 10.7 | 437 | 0.917 |
| M4 GPU (MPS), bs 2048 | 0.095 | 1.535 | 1,163 K | 4.1 | 432 | 0.933 |
| M4 GPU (MPS), bs 4000 | 0.097 | 1.739 | 2,013 K | 7.0 | 433 | 0.908 |
| CPU 10-core, JAX bf16 | 0.208 | 0.588 | 669 K | 16.2 | 385 | 0.914 |



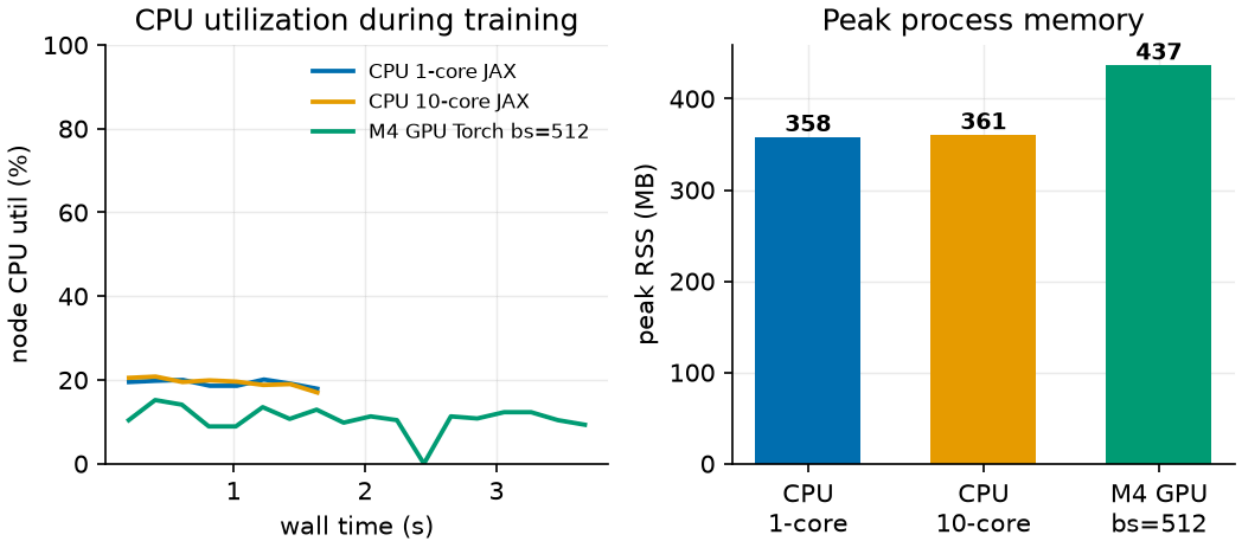
Scaling matrix throughput




Batch-size saturation



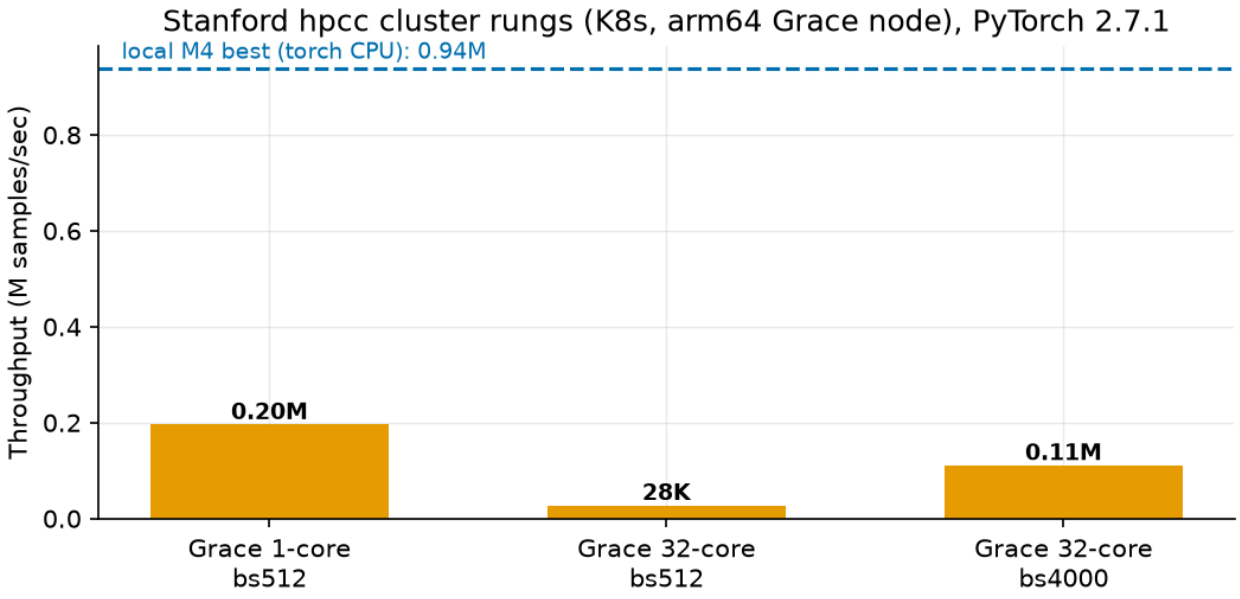

Step time and compile overhead




Node telemetry

Headline deltas: 10 cores over 1 core = **1.00x**; GPU over CPU at batch 512 = **0.42x** (a slowdown); the same GPU at batch 4000 = **5.15x** over itself and **2.1x** over the best CPU run; bf16 on CPU = **0.75x** (regression).

**Cluster validation (measured after VPN access returned).** The identical trainer ran on the Stanford hpcc Kubernetes cluster via the `k8s/hpcc/` manifests: pinned aarch64 PyTorch 2.7.1 wheels in an ephemeral `python:3.12-slim` container (the official images are amd64-only and the worker is a 72-core arm64 Grace-class node), code and dataset delivered as ConfigMaps, 36 s environment bootstrap. Results: 1 core at batch 512 = 196 K samples/s; 32 cores at batch 512 = 27.8 K samples/s, a **7.1x slowdown from adding 31 cores**; 32 cores at batch 4000 = 111 K samples/s. The oversubscription pathology predicted by the local matrix reproduces on a second CPU architecture, more severely: OpenMP barrier costs on sub-millisecond GEMMs scale with thread count, so parallelism is negative-value here (figure `fig5_cluster.png`). The cluster's single NVIDIA GPU is time-shared; the GPU job is queued and reports into the same schema when scheduled.




Cluster rungs

## 5. Bottleneck Diagnosis

**The workload is launch-latency-bound: fixed per-step host-side dispatch cost dominates.** Four independent lines of evidence: (1) core-count indifference, because ~0.1 ms of GEMM per step is below the threshold where thread-pool synchronization pays; (2) GPU step time ~3x CPU step time for identical math, which is the Metal kernel-launch plus host-device sync floor; (3) near-linear throughput scaling with batch size on both backends, the signature of a fixed per-step cost; (4) no other resource is stressed, with node CPU under 25%, GPU memory ~19 MB, RSS 0.44 GB of 16 GB, and IO one 341 KB read per job. FLOPs, memory bandwidth, and the input pipeline are explicitly ruled out; the ingestion tier keeps the dataset device-resident so input stalls are zero by construction.

## 6. Engineering Mitigations and Recommendations

Measured: batch-size scaling to 4000 (5.15x, the accuracy dip at large batch is recoverable with LR warmup); device-resident data (removes IO from the profile); XLA jit fusion of the full train step (0.4 ms CPU steps). Measured and rejected: bfloat16 on M4 CPU, a 0.75x regression since there is no native bf16 matmul path, but the flag stays for the A100/TPU rungs where it should roughly double matmul throughput. Recommended: epoch-level fusion with `jax.lax.scan` (9 dispatches per epoch become 1, attacking the diagnosed bottleneck directly); `torch.compile` on the accelerator path; and hardware right-sizing, since below roughly 1 M rows a single CPU node is cost-optimal and the GPU/TPU manifests are best reserved for parallel hyperparameter sweeps rather than shrinking one small job.

**Cost trade-off.** The best GPU configuration saves ~0.8 s per 400-epoch run over the CPU. At cloud prices (an L4 node at roughly $1/hr vs a comparable CPU node at ~$0.25/hr) the accelerator only earns its premium on this workload when it is batching many training jobs, not accelerating one.

## 7. Status and Next Steps

The SLURM rungs (`train_cpu`, `train_gpu`, `train_multinode`) and the GKE GPU/TPU jobs are provisioned but not yet measured; they were blocked at benchmark time by Stanford VPN access to `hpcc-cluster-49`. They submit unchanged once connectivity returns, and their results drop into the same `benchmarks/results/` JSON schema, figures, and tables.